line-edit-pkg.lisp Manual

File Configuration

This program is written in Common Lisp and provides functionality equivalent to the readline library for Common Lisp environments. It is configured with the following group of programs:

- **line-edit-pkg.lisp** ;; A package that provides line-editing capabilities.
- **history-pkg.lisp** ;; A package that provides history management capabilities.
- **history-repl.lisp** ;; A package that provides REPL (Read-Eval-Print Loop) capabilities.
- **package-util.lisp** ;; A package that provides package-related utility capabilities.
- **print-color-string.lisp** ;; A package that provides xterm256 and color capabilities.
- **support-functions.lisp** ;; A package that provides various supporting functions.

Place these files in any desired directory (hereinafter assumed to be ~/Lisp/line-edit), and place the following files into the ~/.config/line-edit directory:

- **emacs-mode.lisp** ;; A key-binding definition file for Emacs-compatible editing commands.
- **vi-mode.lisp** ;; A key-binding definition file for vi-compatible editing commands.
- **vi-mode-function.lisp** ;; A definition file for editing functions that do not have a one-to-one correspondence with Emacs functions.
- **WordMaster-mode.lisp** ;; A key-binding definition file for WordMaster-compatible editing commands.
- **WordMaster-mode-function.lisp** ;; A definition file for editing functions that do not have a one-to-one correspondence with Emacs functions.
- **set-editor-mode.lisp** ;; A file used to record the editor mode that was used last time.
- **cursor-info.lisp** ;; A descriptive file to define whether the terminal features cursor color modification capabilities.
- **init-repl-prompt.lisp** ;; A descriptive file to configure the prompt settings at startup.
- **registered-message-file.lisp** ;; A message file that supports English, German, and other languages in addition to Japanese.
- **syntax-info-list.lisp** ;; A completion definition file for approximately 800 types of Common Lisp functions and forms.
- **user-info-list.lisp** ;; A descriptive file for defining completion commands that you want to be active right from startup.
- **line-edit-init.lisp** ;; A file intended for writing user-specific initial configuration items.

However, if the following files cannot be loaded at startup, no line-editing capabilities can be used at all. Therefore, the system searches for and loads these specific files strictly in this order: the ~/.config/line-edit/ directory, then the current directory, and finally the home directory. If no configuration is found in any of these locations, the system falls back to a system default editor (defined by the defparameter *default-editor-mode* within line-edit-pkg.lisp; the default configuration is emacs-mode). To prevent confusion when modifying file contents, it is highly recommended to place them exclusively in ~/.config/line-edit.

- emacs-mode.lisp
- emacs-mode.help
- vi-mode.lisp
- vi-mode-function.lisp
- vi-mode.help

- WordMaster-mode-function.lisp
- WordMaster-mode.help
- WordMaster-mode.lisp

Startup Method

Files starting with "compile-" are used by the make command when compiling each package. Place these files in the same directory as the source code files (~/Lisp/line-edit).

Currently, two Common Lisp implementations are supported: SBCL and CLISP. Development is carried out on **Ubuntu 24.04.4 LTS**, with a 13th Gen **Intel Core i7**-13700 × 24 CPU. Although the program has not been verified in other environments, implementation-dependent and OS-dependent features have been carefully designed to be switchable via compile-time options.

After confirming that you can execute your Common Lisp implementation within the ~/Lisp/line-edit directory, if the implementation utilizes the readline library, you must disable it before compiling. SBCL does not use the readline library, but CLISP uses it by default, so disabling it is required. It is convenient to define aliases for SBCL and CLISP as follows:

Bash

```
alias clisp='/usr/bin/clisp -ansi -q -q -disable-readline -i load-repl.lisp -repl -on-error abort'
alias sbcl='/usr/local/bin/sbcl --noinform --load load-repl.lisp'
```

Note that it is better for the implementation to have access to as much memory as possible. For instance, on a machine with 64GB of RAM, it is recommended to add -m 65536MB to the end of the CLISP alias definition (just before the closing single quote '), and --dynamic-space-size 65536 for SBCL. This completes all the required configurations up to this point.

The "Makefile" is the default file for the make command to control the compilation order based on file dependencies.

Running:

Bash

```
make lisp=sbcl
```

will perform the compilation using SBCL, and running:

Bash

```
make lisp=clisp
```

will perform the compilation using CLISP.

Additionally, running:

Bash

```
make lisp=sbcl clean (make lisp=clisp clean)
```

will delete the object code files (.fasl, .fas) generated during compilation. If you

use check-clean instead of clean, the system will only display a list of files to be deleted without actually removing them. After completing the steps above, executing:

Lisp

```
(load "load-repl.lisp")
```

after launching your implementation will start the line-edit-pkg system. However, if you have configured the alias definitions described above, the line-edit-pkg system will start up automatically just by running the implementation's execution command. If the compilation succeeds and you have set up the alias definition to autoload load-repl.lisp, you will see a startup message like the following for SBCL:

Line-Editing Commands Offer Emacs Mode, vi Mode, and WordMaster[1] Mode

Because line-editing commands utilize a mechanism that maps a key sequence for executing a command to an execution function, they can be flexibly modified, just like real Emacs. The editing functions in the core of line-edit-pkg are created to have a one-to-one correspondence with the features of Emacs mode. In most cases, features in vi mode or WordMaster mode commands that do not exist in Emacs mode can be implemented to the extent of combining functions for Emacs mode.

vi-mode-function.lisp

WordMaster-mode-function.lisp

are files that describe additional function definitions required for each respective mode. They are loaded automatically from vi-mode.lisp and WordMaster-mode.lisp. The contents of the emacs-mode.lisp file list definitions that map a key sequence to the

1 WordMaster is an editor that was frequently used in the 1980s during the era of CP/M, an 8-bit OS. Because source materials are scarce, there is a possibility that it has not been fully built out. If you have the manual from that time, please feel free to build it out.

name of the function called when that key sequence is typed, using a function called **global-set-key**. Quoting a part of it, it looks like the following:

Lisp

```
(global-set-key "\\C-@" #'set-mark-command)
(global-set-key "\\C-_" #'set-mark-command)
(global-set-key "\\C-^" #'scroll-down)
(global-set-key "\\C-a" #'beginning-of-line)
(global-set-key "\\C-b" #'backward-char)
(global-set-key "\\C-d" #'delete-char)
(global-set-key "[Delete]" #'delete-char)
(global-set-key "\\C-e" #'end-of-line)
(global-set-key "\\C-f" #'forward-char)
(global-set-key "[Backspace]" #'delete-backward-char)
(global-set-key "[Tab]" #'complete-symbol)
(global-set-key "[Enter]" #'end-input)
(global-set-key "\\C-k" #'kill-line)
```

The function called forward-char, which moves the cursor (point) one character forward, is mapped such that it is called when the Control key and the f key are typed. The same applies to vi-mode.lisp and WordMaster-mode.lisp.

The commands available in each editor mode can be displayed by typing:

Lisp

```
(help-edit)
```

while in each respective editor mode. Depending on the editor mode, it displays the file emacs-mode.help, vi-mode.help, or WordMaster-mode.help. In the case of Emacs mode, the following file is displayed:

```
Command keys:
      C-u [num] Do next command 'num' times (optional, default=4 times).

      C-f Move forward one character.
      M-f Move forward one word.
      M-C-d Move forward down one level of parenthesis.
      M-C-n Move forward across one balanced group of parenthesis.
      M-C-f Move forward across one balanced symbolic-expression.

      C-b Move backward one character.
      M-b Move backward one word.
      M-C-u Move backward out of one level of parenthesis.
      M-C-p Move backward across one balanced group of parenthesis.
      M-C-b Move backward across one balanced symbolic-expression.

      M-p Move to balanced parenthesis.
      M-n Move to next word (like conventional editor).

      C-a Move to beginning of line.
      C-e Move to end of line.
      M-a Move to beginning of text.
      M-e Move to end of text.

      C-p Search history backward (previous history).
      C-n Search history forward (next history).
```

C-v Scroll down one line.
M-v Scroll up one line (or C-^).

C-x < Scroll left.
C-x > Scroll right.

M-< Move to oldest history.
M-> Move to end of history (same as M-k).

C-i Completion (or TAB).
C-s Incremental search.

C-d Delete character (or DEL).
M-d Kill word.
M-C-k Kill symbolic-expression.
C-k Kill line from cursor to end.
M-k Kill whole line and reset search position.

BS Delete backward character (or C-h).
M-BS Backward kill word (or M-C-h).

M-z <c> Kill characters until <c> (zap-to-char).
M-\ Delete spaces and tabs around point.
M-s Delete spaces and tabs around point, leaving one space.

C-t Transpose characters.
M-t Transpose words.
M-C-t Transpose symbolic-expression.

M-(Enclose following symbolic-expression in parenthesis.

C-@ Set region mark (or C-o)
M-@ Mark word (or M-o).
M-C-@ Mark symbolic-expression (or M-C-o).
C-x C-x Exchange point and mark.

C-w Kill-region (cut).
M-w Kill-ring-save (copy).
M-C-w Force next kills to append to previous kill.

C-y Yank (paste from kill-ring).
M-y Paste from kill-ring 2'nd last, 3'rd last,..

C-x r s <R> Copy region to register <R> (<R> = any character).
C-x r i <R> Insert text from register <R>.

C-x i Insert file just before cursor position.
C-x C-f Discard line then insert file.
C-x C-w Write current line to specified file.
C-x C-s Write current line to file.

M-u Convert word to upper-case (upcase-word).
M-l Convert word to lower-case (downcase-word).
M-c Capitalize word.

C-l Redraw line, delete control code.

```
      C-g Abort command.

      C-x u Undo previous changes (or C-_ or M-,).
      C-x U Undo previous undo (or M-.).

      C-x ( Start keyboard macro.
      C-u C-x ( Append to last macro defined.
      C-x ) End Keyboard macro.
      C-x e Call last keyboard macro.
      C-x q Query user during kbd macro execution.
      C-x C-k Edit a keyboard macro.

      C-x = print infomation on cursor position.

      C-q Quote next character.

      <ENTER> end input.

      C-f means press <Ctrl> key then press <f>.
      M-f means press <Meta> key then press <f>.
      Most PC's keyboard, use <Alt> or <Esc> instead of <Meta>.
      If use <Esc>, press <Esc> key and release it, then press <f>.

unkeybinded-functions:
      (self-insert-newline) insert #\Newline.
      (view-register) print registers which contains text.
      (view-register #\R) print contents of register R.
      (save-registers) saves all register to file.
      (restore-registers) restore all registers from file.
      (registers-file) returns current registers file-name.
      (registers-file fname) set registers file to 'fname'.
      (undo-limit) returns current undo limit.
      (undo-limit n) set undo limit to n.
      (blink-paren-deley) blinking time in sec.
      (blink-paren-deley n)   set parenthesis blinking time in sec.
      (blink-paren-just-inserted mode)    blink parenthesis just inserted.
      (blink-paren-just-inserted)   blink parenthesis just inserted ?
      (last-kbd-macro) returns list of last keyboard macro.
      (kbd-macro-query-time n) set how many sec. waiting for C-x q.
      (kbd-macro-query-time) returns how many sec. waiting for C-x q.
      (save-macro fname) saves kbd macro to fname.
      (save-macro) saves kbd macro to (macro-file).
      (load-macro fname) load kbd macro from fname.
      (load-macro) load kbd macro from (macro-file).
      (macro-file fname) set default load/save file for kbd macro.
      (macro-file) returns current load/save file for kbd macro.
```

In this file, C-x represents pressing the Control key while typing the x key, and M-x represents pressing the Meta key while typing the x key. However, since most keyboards do not have a Meta key, the Alt key is used instead. Also, typing the ESC key and then typing the x key has the same meaning as M-x.

Since it is defined to operate as closely as possible to the original editors, please refer to the command manuals of each respective editor for details.

WordMaster is an editor that was frequently used in the 1980s during the era of CP/M, an 8-bit OS. Because source materials are scarce, there is a possibility that it has not been fully built out. If you have the manual from that time, please feel free to build it out.

History Function Supports Both readline and C-shell Styles

Commands that have no functional meaning in a line editor, such as moving to the line above, are mapped to the "move to the previous history" function. If you type C-p (when in Emacs mode) while the prompt line is completely empty, the previous line will be displayed. Typing the Return key as it is will execute the displayed command (function).

If you want to efficiently search for a command entered in the past, typing C-p after entering the first few characters will display the history that matches the prefix of the entered characters. Typing C-p again in that state will display the next oldest history whose prefix matches. This can be repeated until the very beginning of the saved history is reached. For example, if you have entered (pushd :line-edit-pkg) in the past, and prior to that input, you have entered (pushd :support-functions):
Typing C-p with the following state entered:

```
Lisp
      (pushd
```

Will first display:

```
Lisp
      (pushd :line-edit-pkg)
```

And typing C-p once more in that exact state will then display:

```
Lisp
      (pushd :support-functions)
```

Typing the Return key while it is displayed will execute (pushd :support-functions). The displayed history can be freely modified using line-editing commands. By default, the history stores up to 1024 entries. This size is defined by a parameter named ***hist-size*** within **history-pkg.lisp**. As long as memory permits, you can configure this to any large value. History information that exceeds the maximum storage count is discarded sequentially from the oldest history.

At the end of a REPL session, all history is automatically saved to a file, and the saved history is reloaded at the start of the next session.
The history search via C-p mentioned above is the readline style, but history operations using the C-shell style can also be performed. This C-shell style history operation consists of commands starting with an **! (exclamation mark)**.
Typing:

```
Bash
      !!
```

and then typing the Return key re-executes the contents of the line executed immediately before.
Typing:

```
Bash
```

```
!h
```

and then typing the Return key displays up to 25 of the most recently entered history entries along with their history numbers. If the history number of the input you wish to re-execute is 1011, you type:

```
[clisp:line-edit-pkg #1025]> !h
1000:(pushd :line-edit-pkg)
1001:(pushd :line-edit-pkg)
1002:(pushd :line-edit-pkg)
1003:(pushd :line-edit-pkg)
1004:(pushd :line-edit-pkg)
1005:(pushd :line-edit-pkg)
1006:(ls)
1007:(pushd :support-functions)
1008:(exchgd)
1009:(help 'ls)
1010:(help-f 'ls)
1011:(helpf 'ls)
1012:(select-language :en)
1013:(popd)
1014:(pushd :support-functions)
1015:(cd)
1016:(pushd :line-edit-pkg)
1017:(pushd :support-functions)
1018:(help-history)
1019:(help-complete)
1020:(select-language :ja)
1021:(help-complete)
1022:(cd)
1023:(pushd :line-edit-pkg)
1024:(pushd :line-edit-pkg)
T
[clisp:line-edit-pkg #1025]>
```

Bash

```
!1011
```

and then type the Return key. In this C-shell style history operation, you can also directly edit the contents of the history storage file using a registered editor. This configuration is defined by the parameter ***favorite-editor**** within history-pkg.lisp. **Currently, "vi" is configured**.

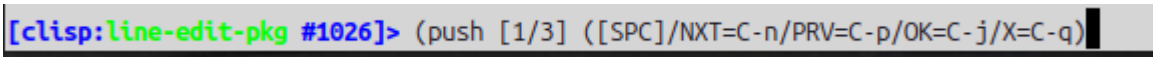
The history functions in the C-shell style are:

```
!n              re-do n'th history.
                same as (do-history n).
!-n             re-do current minus n'th history.
```

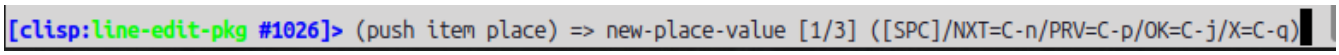
```
                    same as (do-history -n).
      !!            re-do last input. same as !-1.
                    same as (do-history).
      !do m n       do history from m to n.
                    same as (do-history m n).
      !history      print all history.
                    same as (history).
      !hist n       print last n history.
                    same as (history n).
      !h            print last histories.
                    same as (last-histories).
      !r or !read           read history from file.
                    same as (read-history).
      !w or !write  write history to file.
                    same as (write-history).
      !c or !clear  clear current history, not a file.
                    same as (clear-history).
      !edit               edit and re-read history file.
                    same as (edit-history).
      !ed n               edit and eval n'th history.
                    same as (edit-history n).
      !ed -n        edit and eval current minus n'th history.
                    same as (edit-history -n).
      !e            edit and eval last history.
                    same as (edit-history -1).

unkeybinded-functions:
      (restore-history)           read history's history and history file.
      (reset-history) reset history # to 0, then read history.
      (history-size) returns current history buffer size.
      (history-size n) set history buffer size to n.
      (last-histories) returns current default number of prints.
      (last-histories n) set default number of prints. see !h.
      (history-with-number t) print history with number (default).
      (history-with-number nil)    without number.
      (history-file fname) set history file to 'fname'.
      (history-file) returns current history-file's name.
      (history-buffer fname) set history's buffer to 'fname'.
      (history-buffer) returns current history buffer's name.

      (help-edit) show current editor mode commands.

This list can be displayed by typing:

Lisp
      (help-history)

The "n" in the table above represents a history number (integer).
```

Completion Function with Registered Grammar Information for Approximately 800 Types of Standard Common Lisp Functions and Macros.

The completion function is a capability that, when the Tab key (default setting) is typed after entering the first few characters, presents candidates whose prefix matches the entered characters. Typing the confirmation key (definable for each editor mode; the default setting in Emacs mode is C-j) confirms the selection. If the presented candidate is not the desired one, typing the Tab key again as it is will present another candidate one after another. The same line is overwritten to display the new candidate.



Candidates can be selected from **"short candidates"** or **"long candidates"**. The short candidates and long candidates **switch each time the Space key is typed**.



In the case of approximately 800 types of **standard Common Lisp functions**, selecting **"long candidates"** with the Space key displays a string accompanied by **argument and return value information**. As displayed on the completion candidate selection line, confirm with **C-j**. To move to the next completion candidate, type **C-n** (in addition to the Tab key); to return to the previous completion candidate, type **C-p**; and to cancel the selection of completion candidates, type **C-q**. These choice keys can be freely changed according to the currently selected editor mode.

The configured keys are reflected and displayed on the completion candidate selection line. Although not displayed, multiple keys may be assigned in some cases. What the actual key assignments are can be verified by checking the mapping between key sequences and execution functions in complete-symbol-set-key within the definition file of each editor mode.

Lisp

```
(complete-symbol-set-key "\\C-p" :previous-candidate) ;; Return to the previous candidate.
(complete-symbol-set-key "\\M-\\[A" :previous-candidate) ;; (↑) up arrow (HHKB=Fn + [).

(complete-symbol-set-key "\\C-n" :next-candidate) ;; Move to the next candidate.
(complete-symbol-set-key "\\M-\\[B" :next-candidate) ;; (↓) down arrow (HHKB=Fn + /).
(complete-symbol-set-key "\\C-i" :next-candidate) ;; Tab.

(complete-symbol-set-key "\\C-j" :current-candidate) ;; Confirm current candidate.
(complete-symbol-set-key "." :current-candidate) ;; [,][.][/]=[Short][Standard][Long]
(complete-symbol-set-key "\\C-." :current-candidate) ;; [,][.][/]
(complete-symbol-set-key "\\M-." :current-candidate) ;; [,][.][/]

(complete-symbol-set-key "\\C-s" :short-candidate) ;; Confirm as short candidate.
(complete-symbol-set-key "," :short-candidate) ;; [,][.][/]
(complete-symbol-set-key "\\C-," :short-candidate) ;; [,][.][/]
(complete-symbol-set-key "\\M-," :short-candidate) ;; [,][.][/]

(complete-symbol-set-key "\\C-l" :long-candidate) ;; Confirm as long candidate.
(complete-symbol-set-key "/" :long-candidate) ;; [,][.][/]
(complete-symbol-set-key "\\C-/" :long-candidate) ;; [,][.][/]
(complete-symbol-set-key "\\M-/" :long-candidate) ;; [,][.][/]

(complete-symbol-set-key "r" :redraw) ;; Redraw the current line and
(complete-symbol-set-key "\\C-r" :redraw) ;; delete control codes other than #\Newline.
(complete-symbol-set-key "\\M-r" :redraw)

(complete-symbol-set-key "q" :cancel) ;; Cancel completion.
(complete-symbol-set-key "\\C-q" :cancel)
(complete-symbol-set-key "\\M-q" :cancel)

;; '(:previous-candidate :next-candidate :current-candidate :short-candidate :long-
candidate :redraw :cancel)
;;(setf *complete-symbol-used-key-def* '("C-p" "C-n" "." "," "/" "C-r" "q"))
(setf *complete-symbol-used-key-def* '("C-p" "C-n" "C-j" "C-s" "C-l" "C-r" "C-q"))
```

The keys actually displayed as completion candidate choices are defined in the last

line by:

```
Lisp
      (setf *complete-symbol-used-key-def* …)
```

However, for example, the key sequence for "returning to the previous candidate" is assigned to the key sequence generated when typing the Up arrow key in addition to C-p. Therefore, you can also "return to the previous candidate" with the Up arrow key (Fn + [on HHKB)[2].

When there are multiple completion candidates to display, a priority value along an exponential decay curve is calculated based on the number of times that completion candidate was selected in the past and the date and time it was last selected, and they are displayed in descending order of priority. The exponential decay curve is a curve obtained by calculating the formula:

$$\text{Score} = \text{Count} \times 1/2^{(\Delta T/\tau)}$$

using the parameters:

- Count: Number of adoptions.
- ΔT: Difference between the current time and the last reference time.
- τ (tau): Half-life.

The number of adoptions, the current time, and the difference from the last reference time are automatically set by line-edit-pkg. The half-life (tau) is the time (in seconds) until the value representing the calculated priority decays to 1/2.

If a large value is set, the priority becomes harder to rise, but once risen, the priority becomes harder to fall. Conversely, if a short time is set, the priority rises and falls sensitively.

When you want to maintain a stable ranking, the recommended value for the half-life is about 2 weeks (= 14 days × 24 hours × 60 minutes × 60 seconds). Conversely, when you want the ranking to rise and fall sensitively, the recommended value for the half-life is about 3 hours (= 3 hours × 60 minutes × 60 seconds). This configuration is set in the initial configuration file line-edit-init.lisp attached as a sample by:

```
Lisp
      (line-edit-pkg:half-life-hour …)
```

Please rewrite it according to your preference.

Information regarding the number of confirmations and the final time of confirmation for completion candidates is automatically saved at the end of the REPL session. Therefore, the fluctuated priorities of completion candidates are carried over to the next REPL session.

For user packages that existed at startup, each time you move (= in-package) to that package, external symbols, internal symbols, and inherited symbols are retrieved and registered as completion candidates on the spot each time. However, since it is not known when these candidates will be added or deleted, their priorities are recorded and fluctuate during the REPL session, but the information is not saved at the end

2 Whether special keys such as arrow keys and function keys can return keycodes as special escape sequences when typed depends on the terminal. Generally, it is supported if DEC VT510 or higher is used and mode 1 of extended sequences is implemented.

of the REPL session.

Method of Defining the REPL Prompt

The REPL prompt can be freely changed with an intuitive definition. You simply specify the elements you want to display in order to the function **set-prompt-element** inside **history-pkg**. The elements that can be specified are:

- :current-package: Current package name (if it has a nickname, returns the nickname).
- :original-package-name: Returns the current package name. Even if it has a nickname, returns the original name.
- :not-cl-user: Returns the package name only when it is not the cl-user package.
- :date: Today's date in ISO format (YYYY-MM-DD).
- :time: Current time in 24-hour format (HH:MM:SS).
- :time12: Current time in 12-hour format (HH:MM{am,pm}).
- :absolute-dir: Current directory (absolute path).
- :working-dir: Current directory (relative path).
- :working-dir-name: Current directory name only.
- :history-number: History number.
- :os-type: Type of OS.
- :host-name: Host name / machine name.
- :machine-type: CPU type.
- :lisp-type: Implementation name.
- :lisp-version: Version number of the implementation.
- :heap-size: Size of the heap area being used (MB). SBCL only.
- "string": A string.
- :change-to-red: Changes the subsequent display to red.
- :change-to-green: Changes the subsequent display to green.
- :change-to-blue: Changes the subsequent display to blue.
- :change-to-cyan: Changes the subsequent display to cyan.
- :change-to-magenta: Changes the subsequent display to magenta.
- :change-to-yellow: Changes the subsequent display to yellow.
- :change-to-gray: Changes the subsequent display to gray.
- :change-to-black: Changes the subsequent display to black.
- #'func: The result returned by the function func.

By writing prompt configuration information inside **init-repl-prompt.lisp**, the specified prompt information will be reflected at startup.

Inside init-repl-prompt.lisp, the following configuration is written as a sample:

Lisp

```
(package-util:set-package-name-case :downcase) ;; Display the package name in lowercase.
(history-pkg:set-prompt-attributes :color 'blue :bold t) ;; Specification of initial color
                                                         ;; attributes for the whole.
(history-pkg:set-prompt-element
       "["           ;; Display "[".
       :lisp-type    ;; Implementation name.
       ":"           ;; Display ":".
       #'print-color-string:change-to-green ;; Green from here on.
       :not-cl-user  ;; Displays the package name only when the current package is not [:cl-user].
                     ;; The display color is green.
       #'print-color-string:change-to-blue ;; Blue again from here on.
       #+sbcl "("    ;; In the case of SBCL, display "(".
       #+sbcl #'print-color-string:change-to-red ;; Red from here on.
       #+sbcl :heap-size    ;; In the case of SBCL, display the heap size.
       #+sbcl #'print-color-string:change-to-blue ;; Blue again from here on.
       #+sbcl ")"    ;; In the case of SBCL, display ")".
       ;;" "         ;; Display a space (" ").
       ;;:date       ;; Date in ISO format (YYYY-MM-DD)
       ;;" "         ;; Display a space (" ").
       ;;:time       ;; Display the time in 24-hour system (HH:MM:SS).
       " #"          ;; Displays a (" #").
       :history-number "]> " ;; Color and attribute specifications for characters are reset here.
) ;; end history-pkg:set-prompt-element
```

With this configuration, in the case of SBCL, it is displayed as follows at startup: Since the overall color attribute is set to blue, bold, "[" is displayed in blue characters, then the implementation name (:lisp-type) is displayed as 'sbcl', the string ':', no package name display because it is the Common-Lisp-User package at startup, if the implementation is SBCL the string '(' is displayed, the subsequent display color is changed to red, if the implementation is SBCL the heap size is displayed (SBCL unique function), the subsequent display color is changed to blue again, if the implementation is SBCL the string ')' is displayed, the string ' #' is displayed, then the history number is displayed, and finally the string ']> ' is displayed to finish.

The above is executed every time the REPL prompt is updated. Therefore, if the keyword :time is written, the latest hour, minute, and second will be displayed every time.

If the prepared keywords are insufficient, functions can be specified in the form of **#'func**, so it can be customized even more freely by preparing a function that returns your preferred result.

Package Migration and Directory Stack Capabilities

This environment features robust support for navigating between Common Lisp packages and maintaining a directory style structure for packages:

(pushpd :package-name): Pushes the current package onto the package directory stack and migrates to the specified package.
(poppd): Pops the top package off the directory stack and moves back to it.
(rotdu): Rotates the directory stack upward.
(exchgd): Swaps the order of the top two packages in the directory stack.
(dirs): Displays the current status and contents of the package directory stack.
(ls): Lists the external, internal, and inherited symbols within the current package.

If the configuration (auto-show-dirs t) is enabled, the system will automatically execute the (dirs) function every time a package migration occurs. The ls function provides several keyword parameters, allowing you to filter and display only the specific information you need:

```
:string "string"     ;; Displays only symbols containing the specified "string".
                     ;; The default value is "" (= show all).
:package 'package'   ;; Accepts a package string or a package object. The default value is
                     ;; *package*.
:external [t/nil]    ;; Defines whether to display external symbols. Default is t.
:internal [t/nil]    ;; Defines whether to display internal symbols. Default is nil.
:inherited [t/nil]   ;; Defines whether to display inherited symbols. Default is nil.
:verbose [t/nil]     ;; Displays additional attributes and properties of the symbols.
                     ;; Default is t.
:quiet [t/nil]       ;; Suppresses the printing of title headers for each attribute group.
                     ;; Default is nil.
```

For example, executing the command as:

Lisp
```
(ls :string "push" :external t)
```

will filter the output to display only the external symbols that begin with or contain the string "push".

```
[sbcl:line-edit-pkg(53.9MB) #1025]> (ls :string "push" :external t)
External symbols in line-edit-pkg:
        (Function) push-command-trace-list
        (Function) push-keyboard-macro
        (Function) push-kill-ring
        (Function) push-redo-info
        (Function) push-undo-info
5 symbols
Total: 5 symbols
[sbcl:line-edit-pkg(55.6MB) #1026]>
```

Before external symbol names, the type of the symbol (such as a function, macro, or constant) is also explicitly displayed. Giving a nil value to the :verbose keyword parameter will omit these type indicators from the output.

```
[sbcl:line-edit-pkg(56.5MB) #1026]> (ls :string "push" :external t :verbose nil)
External symbols in line-edit-pkg:
        push-command-trace-list
        push-keyboard-macro
        push-kill-ring
        push-redo-info
        push-undo-info
[sbcl:line-edit-pkg(58.8MB) #1027]>
```

In addition to migrating via the package directory stack styles described above, moving between packages can also be performed traditionally using the standard form:

Lisp

```
(in-package :package-name)
```

Even when the current package is altered using this traditional style, the underlying REPL engine instantly detects the migration and automatically updates the prompt to reflect the new package name.

In terms of operation, it is the same as:

Lisp

```
(cd :package-name)
```

Main Messages from the Program Support Multiple Languages

The main messages from the program can be freely switched to be displayed in languages prepared through advance translation by AI. The default is Japanese messages, but currently, in addition to Japanese (:ja), messages are prepared for:

- English (:en)
- German (:de)

- French (:fr)
- Simplified Chinese (:zh-hans)
- Traditional Chinese (:zh-hant)
- Korean (:ko)
- Zulu (:zu)

Switching is performed by:

```
Lisp

      (select-language :en)
```

which switches to English. If these configurations are written inside the system-wide initialization file "line-edit-init.lisp", the settings will be completed at startup. It is also possible to switch during a REPL session.

Ubuntu was founded by Mark Shuttleworth from South Africa, and out of respect for the word's meaning, which includes "compassion towards others" and "humanity," **Zulu**, which is spoken around South Africa, is also supported. It is said that the underlying philosophy is "a person is a person through other persons."

Contact / Feedback: mail2daigo@gmail.com